Adaptive Sketching for Tensor Network Contraction

Anonymous Submission

Abstract

带开放腿的张量网络并不总是以标量收缩结束。在误差评估、可视化与跨系统接口等场景中，下游真正需要的是完整的稠密输出张量。此时，问题的关键不在于改写输出，而在于如何在保持输出边界不变的前提下，降低生成该输出所需的内部收缩代价。本文研究一种面向完整物化的近似框架：先对输出索引空间做分块，再在交互图上构造划分树，并沿树递归维护分隔状态。方法始终保持输出边界显式存在，只对内部接口做近似；在内部合并处，进一步通过基于算子访问的随机化 sketch 压缩，避免先显式形成膨胀接口、再做截断的中间瓶颈。

实验围绕两条主线展开。第一，我们在 ring、PEPS-like 与随机连通图上做尺度扫描，考察本文方法相对精确完整收缩的 frontier。结果表明，在 exact 可得的比较区间内，本文方法的相对误差始终低于 7.5%，而运行时间相对 Exact-Path 约有 7.4× 到 $3.5 \times 10^3\times$ 的加速；进一步地，在 Ring-8 到 Ring-14、PEPS 2×4 到 4×4 的扫描中，本文方法完成了全部测试点，而 Exact-Path 分别从 Ring-10 与 PEPS 3×4 开始在给定时间预算下不可用。第二，我们进一步分析了关键设计的作用。结果表明，固定秩方案更快但误差明显更大；cache reuse 的作用在更大规模时更加清楚；implicit merge sketch 在很小的 exact-available 实例上收益并不总是稳定，但在更大的合并接口上开始表现出正向作用。综合来看，本文支持的不是普适的运行时间优势，而是：在完整物化任务下，保持输出边界的自适应分隔状态近似，配合基于 sketch 的合并压缩，能够形成更稳健、并且随规模增大更有价值的精度–时间折中。

1 引言

张量网络为高维数组的表示与计算提供了紧凑而有结构的形式，已经广泛用于量子多体计算、统计物理、图模型推断以及张量算法等方向 [15]。围绕张量网络收缩，已有工作主要关心三类问题：如何设计更好的收缩顺序，如何控制中间状态规模，以及如何通过中间状态压缩来降低近似收缩的代价 [14, 5, 11, 7, 10, 16, 6, 13]。现有张量网络收缩研究大多围绕标量收缩、局部可观测量计算或概率推断中的局部查询展开，而非显式生成保留开放腿后的完整输出张量 [15, 3, 16, 9]。相比之下，一旦要求对若干开放腿对应的全部配置进行物化，输出空间和中间收缩状态都会迅速膨胀，从而带来显著的计算与存储开销 [11]。本文关注的正是这一类带开放腿张量网络的完整稠密输出计算问题。

这一问题与通常的近似收缩有一个根本差别：当完整输出本身就是任务目标时，算法不能将其替换为压缩表示，也不能仅返回少量局部结果。因此，我们关注的问题不是改变输出形式，而是在保持完整输出要求不变的前提下，降低生成这一输出的计算开销。若把端到端时间写成

$$T_{\text{total}} = T_{\text{contract}} + T_{\text{emit}},$$

我们将 T_{contract} 理解为生成目标输出之前的内部计算开销，即网络内部的收缩、合并及相关中间状态处理；将 T_{emit} 理解为把这些结果显式物化为完整输出张量的开销。由于完整输出本身必须被生成，T_{emit} 往往不可回避，因此方法设计的主要优化对象通常是 T_{contract}。

基于这一观察，本文研究一种面向完整物化的近似收缩框架。基本思路是：在网络交互图上构造一棵划分树，并沿树递归维护各个子网络的局部接口；在整个过程中，输出边界始终保持显式，只对内部接口做近似。于是，本文近似的不是最终输出对象，而是“为了得到该输出而必须经过的内部中间状态”。这也是本文与直接压缩输出端的方法，以及以标量收缩为目标的方法之间最本质的区别。

从表示上看，每个子树状态都可以理解为一个从输出侧到边界侧的接口矩阵；本文用低秩因子化在树上递归传播这一状态，并在每次合并后重新压缩，以抑制中间维度增长。进一步地，针对内部节点合并时容易出现的秩乘积膨胀，本文在 ASTNC 中引入基于算子访问的随机化 sketch 压缩，不显式形成完整的合并接口，而是直接恢复其主导子空间。因此，本文的核心并不是某一个孤立的局部技巧，而是把输出分块、划分树递归、分隔状态近似与 sketch 驱动的合并压缩组织成一条统一的完整收缩流程。在这条流程里，输出语义保持不变，近似只服务于降低内部收缩成本。

实验围绕两组问题展开。第一，相对精确完整收缩，ASTNC 在尺度扩展过程中能否保持清楚的误差–时间折中，并在更大规模上继续保持可行；第二，固定秩、缓存、选择性近似和 implicit merge sketch 等内部设计到底分别起什么作用。结果表明：在 exact 可得的比较区间内，ASTNC 在 ring、PEPS-like 与随机连通图上都保持了较低误差，并相对 Exact-Path 给出数量级加速；在进一步的 frontier 扫描中，ASTNC 完成了全部测试点，而 Exact-Path 在更大的 ring、PEPS-like 与随机图实例上先后跨过 timeout 边界；而消融分析进一步表明，固定秩方案更接近“更快但更不准”的一端，cache reuse 在更大规模时带来清楚收益，而 implicit merge sketch 的优势与规模相关：在很小实例上收益并不总是稳定，但在更大的合并接口上开始体现出正向作用。因此，本文支持的结论是：保持输出边界的自适应内部接口近似，配合基于 sketch 的合并压缩，能够在完整物化任务下给出更稳健、并且在更困难实例上更有价值的精度–时间折中。

本文的贡献可以概括为三点：

- 提出一个面向开放腿张量网络完整物化的近似收缩设定，将优化重点从“近似收缩值”转向“在保持输出边界语义不变的前提下降低内部收缩代价”。
- 提出一种基于划分树递归的分隔状态框架，并在

ASTNC 中引入基于算子访问的 implicit merge sketch，以降低内部合并阶段的大接口压缩开销。

- 通过 exact-reference comparison、尺度 frontier 与内部消融说明：ASTNC 能够在完整物化任务中形成更稳健的精度–时间折中；其中，自适应秩选择与 cache reuse 提供了稳定收益，而 sketch 压缩的优势随着合并接口规模增大而开始显现。

2 相关工作

精确张量网络收缩。 精确张量网络收缩的经典研究主要围绕收缩顺序、收缩树、树宽以及切片策略展开。这一路线的核心结论很明确：真正决定复杂度的往往不是单个张量本身，而是中间状态的规模，以及由此诱导出的空间和时间开销 [14, 5, 11, 7]。在此基础上，index slicing 又进一步说明，当一次完整收缩难以承受时，可以把部分自由度切出来，转化为许多更小的子任务来并行执行或分批完成 [10]。这些工作为本文提供了两个直接启发：一是困难首先来自中间分隔结构的膨胀；二是按块组织计算可以成为完整物化过程中的重要系统手段。不过，这一方向的主要目标仍然是精确收缩，而不是在完整输出仍需显式生成的前提下进行近似优化。

近似收缩与中间状态压缩。 当精确收缩代价过高时，近似张量网络方法通常通过压缩中间对象来控制复杂度。在规则拓扑上，TRG 等工作已经表明：面对不可承受的中间状态，能否做出有效截断，是近似收缩能否成立的关键 [12]。近年来，这一思路逐渐从规则晶格走向一般图结构。Pan 等人提出了适用于任意连通结构的一般性近似收缩算法 [16]；Gray 和 Chan 将近似直接纳入收缩规划过程，使压缩与收缩顺序能够联合优化 [6]；Ma 等人则发展了面向一般图的 density-matrix 风格近似框架 [13]。这些工作说明，一般图上的近似收缩已经可以系统地开展。相比已有面向一般图的近似收缩方法，本文关注的是开放腿张量网络的完整物化问题。由于输出部分需要被完整生成，算法不能改变其形式，因此更合理的做法是在保留输出部分的同时，对网络内部的接口状态进行近似处理。

随机化低秩近似与 sketch。 本文的方法也借鉴了随机化数值线性代数与 sketching 的思想。随机化低秩近似表明，在许多大规模问题中，可以先用随机投影快速捕捉主导子空间，再在较小的子空间内做稳定分解，而不必一开始就构造昂贵的精确分解 [8]。更一般地，Count Sketch 与 Tensor Sketch 等方法展示了随机线性映射在高维对象压缩中的作用 [1, 17]。非常近期的工作也开始直接讨论张量网络收缩中的 sketch 近似 [9]。不过，本文并不把自己的贡献表述为一个普适的 sketch 系统；更准确地说，随机化压缩在这里是分隔状态框架中的实现工具之一，其作用是降低内部接口压缩的代价，而不是改变本文的问题设定。

与图模型消去方法的关系。 从算法结构上看，本文也与图模型中的 bucket elimination 和 mini-bucket 有相通之处 [2, 4]。这些方法同样强调：真正向上层传播的不是局部对象的全部细节，而是与外部仍然相关的接口信息；当接口过大时，就需要在中间层面做受控近似。我们把这一思想放在开放腿张量网络的完整物化任务下重新组织，并把输出边界保持视为方法定义的一部分。

3 方法

3.1 整体思路

设 $\mathcal{N}$ 为一个带开放标签集合 $\mathcal{O}$ 的张量网络，内部收缩标签集合记为 $\mathcal{I}$。将所有内部标签收缩后，可得到完整输出张量

$$X = \mathrm{Mat}(\mathcal{N}) \in \mathbb{R}^{d_1 \times \cdots \times d_{|\mathcal{O}|}}.$$

相应地，端到端时间可写为

$$T_{\text{total}} = T_{\text{contract}} + T_{\text{emit}},$$

其中 T_{contract} 表示内部收缩与近似本身的代价，T_{emit} 表示将结果写回最终输出缓冲区的代价。在完整收缩场景下，T_{emit} 通常难以避免，因此真正值得优化的主要是 T_{contract}。基于这一判断，本文把近似严格限制在内部接口上，而不直接压缩输出边界。

整条方法主线可以概括为三步：

1. 在网络交互图上构造一棵二叉划分树，把一般图上的收缩过程写成一系列二叉合并；
2. 对每个子树，用一个从输出侧到边界侧的局部接口来概括它对外仍然可见的信息；
3. 沿树递归合并这些局部接口，并在每次合并后重新压缩，以控制中间状态规模。

3.2 划分树与分隔状态

为了统一组织一般图上的递归收缩，我们在张量网络的交互图上构造一棵二叉划分树（partition tree）。树上的每个节点 v 对应一个子网络 $\mathcal{N}_v$；叶节点对应单个原始张量，内部节点对应两个子网络的合并，根节点对应整个网络。

对任意节点 v，我们关心三类标签：

- **输出标签** $\mathcal{O}_v$：该子树最终仍通向输出侧的标签；
- **边界标签** $\mathcal{B}_v$：该子树与外部其余网络连接所需保留的标签；
- **切割标签** $\mathcal{C}_v$：当 v 由两个孩子合并而成时，连接左右孩子并在本层被消去的标签。

根节点不再与外部网络相连，因此其边界标签为空；叶节点不再继续拆分，因此没有切割标签。

给定输出块 Ω_b 后，我们将节点 v 的分隔状态定义为

$$M_v \in \mathbb{R}^{d_{\text{open}}(v) \times d_{\text{bnd}}(v)},$$

其中

$$d_{\text{open}}(v) = \prod_{\ell \in \mathcal{O}_v} d_\ell, \qquad d_{\text{bnd}}(v) = \prod_{\ell \in \mathcal{B}_v} d_\ell.$$

这里 d_ℓ 表示标签 ℓ 在当前输出块中的局部维度；若该标签未被当前块限制，则 $d_\ell = d_\ell$。矩阵 M_v 的行对应输出侧配置，列对应边界侧配置，因此它刻画了当前子树在输出块 Ω_b 下对父节点可见的局部行为。

直接显式存储 M_v 往往代价过高，因此我们采用低秩因子形式

$$M_v \approx A_v\big(B_v\big)^\top,$$

其中

$$A_v \in \mathbb{R}^{d_{\text{open}}(v)\times r_v}, \qquad B_v \in \mathbb{R}^{d_{\text{bnd}}(v)\times r_v},$$

r_v 是节点 v 的接口秩。这种写法把输出侧与边界侧显式分开，也清楚地说明了本文的近似位置：输出边界保持不变，近似只发生在内部接口状态上。

3.3 树上的递归合并

在定义了分隔状态之后，整棵划分树上的计算就可以写成统一的递归。

叶节点。 对叶节点 v，其子网络只包含一个原始张量。在给定输出块后，我们先将与当前块相关的开放标签限制到局部索引范围，再把张量重排成

$$T_v \in \mathbb{R}^{d_{\text{open}}(v)\times d_{\text{bnd}}(v)}.$$

若不做近似，则 T_v 就是该叶节点的精确分隔状态；若做近似，则对 T_v 做低秩压缩，得到

$$M_v \approx A_v\big(B_v\big)^\top.$$

内部节点。 设内部节点 v 的两个孩子为 v_L 和 v_R，其分隔状态分别为

$$M_{v_L} \approx A_{v_L}\big(B_{v_L}\big)^\top, \qquad M_{v_R} \approx A_{v_R}\big(B_{v_R}\big)^\top.$$

父节点的构造可分为三步。

(i) 合并输出侧。 父节点的输出标签满足

$$\mathcal{O}_v = \mathcal{O}_{v_L} \cup \mathcal{O}_{v_R}, \qquad \mathcal{O}_{v_L} \cap \mathcal{O}_{v_R} = \varnothing.$$

因此，父节点的输出侧配置空间是左右孩子输出侧配置空间的笛卡尔积。在因子层面，这一步对应于将两个孩子的输出侧因子做外积，等价地可写成 Kronecker 积

$$A_{\text{mat}} = A_{v_L} \otimes A_{v_R},$$

其列数为 $r_{v_L} r_{v_R}$。

(ii) 收缩切割标签，保留边界标签。 设父节点保留的边界标签集合为 $\mathcal{B}_v$，本层被消去的切割标签集合为 $\mathcal{C}_v$。则父节点未压缩的边界侧因子可以看成对两个孩子的边界侧因子沿 $\mathcal{C}_v$ 做收缩：

$$B_{\text{mat}} = B_{v_L} \times_{\mathcal{C}_v} B_{v_R}.$$

直观地看，这一步完成了“该消去的内部边被消去，该保留给父节点的边界仍然保留”。

(iii) 重新压缩。 经过前两步后，父节点得到未压缩表示

$$\widetilde{M}_v = A_{\text{mat}} B_{\text{mat}}^\top.$$

由于其中的中间列数已经膨胀到 $r_{v_L} r_{v_R}$，若不立即重压缩，秩会在树上迅速增长。因此，我们对 $\widetilde{M}_v$ 再做一次低秩压缩，写回统一的分隔状态形式

$$M_v \approx A_v\big(B_v\big)^\top.$$

3.4 自适应秩选择

在分隔状态递归中，每次叶节点构造和内部节点合并之后，都要对当前接口重新压缩，以防止秩在树上不断膨胀。最直接的做法是为所有节点统一指定一个固定秩 r。这种方案实现简单，但它默认划分树上不同节点的接口难度大致相同；这一假设在实际中通常并不成立。

不同节点对应的子树规模、边界结构、当前输出块下的局部自由度以及谱衰减形状都可能差别很大。有些节点本身较简单，较低秩已经足够；另一些节点则包含更复杂的内部耦合，若仍采用统一秩预算，就会过早丢失关键信息。因此，固定秩往往会同时带来两类问题：在简单节点上保留了多余通道，在困难节点上又压得不够。

基于这一观察，本文在每个待压缩节点上按局部误差容忍度自适应选择秩，而不是预先指定统一目标秩。设某个待压缩接口矩阵为

$$M \in \mathbb{R}^{m\times n},$$

其奇异值分解为

$$M = U\Sigma V^\top, \qquad \Sigma = \operatorname{diag}(\sigma_1, \sigma_2, \ldots), \qquad \sigma_1 \ge \sigma_2 \ge \cdots \ge 0.$$

若取其秩-r 截断近似

$$M_r = U_r \Sigma_r V_r^\top,$$

则 Frobenius 范数下的相对误差为

$$\frac{\|M - M_r\|_F}{\|M\|_F} = \left(\frac{\sum_{i>r}\sigma_i^2}{\sum_i \sigma_i^2}\right)^{1/2}.$$

因此，给定容忍度 τ，我们选择满足

$$\left(\frac{\sum_{i>r}\sigma_i^2}{\sum_i \sigma_i^2}\right)^{1/2} \le \tau$$

的最小秩 r。在实现中，叶节点与内部合并节点可分别使用不同的容忍度，记为 τ_{leaf} 与 τ_{merge}。

这种写法的好处是，它把“每个节点保留多少通道”转化成“每个节点允许损失多少相对能量”。谱衰减快的简单节点会自动停在较低秩，而谱衰减慢的困难节点会自动保留更多通道。因此，自适应秩选择的关键不在于简单增大平均秩，而在于按局部难度重新分配秩预算。

3.5 基于 sketch 的随机化压缩

仅靠自适应秩选择还不够。在内部节点处，若左右孩子的秩分别为 r_L 和 r_R，则父节点在重压缩之前自然会出现

一个秩乘积 $r_L r_R$。如果先显式构造合并后的大接口矩阵，再对它做普通低秩分解，就会遇到“先膨胀，再压回去”的中间瓶颈。为此，本文在合并节点上进一步采用基于 sketch 的随机化压缩：不显式收缩完整的接口矩阵，而是通过算子访问直接恢复其主导子空间 [8, 9]。

设合并后待压缩的父节点接口为

$$\widetilde{M} \in \mathbb{R}^{m\times n}.$$

在显式路径中，它可以写成

$$\widetilde{M} = A_{\text{mat}} B_{\text{mat}}^{\top},$$

其中 A_{mat} 来自输出侧因子的组合，B_{mat} 来自边界侧因子沿切割标签的收缩。随机化压缩真正需要的不是 $\widetilde{M}$ 的全部条目，而是对线性算子

$$x \mapsto \widetilde{M}x, \qquad y \mapsto \widetilde{M}^{\top} y$$

的访问。在本文的合并结构下，这两个乘法可写为

$$\widetilde{M}x = A_{\text{mat}}\big(B_{\text{mat}}^{\top}x\big), \qquad \widetilde{M}^{\top}y = B_{\text{mat}}\big(A_{\text{mat}}^{\top}y\big).$$

因此，整个压缩过程都可以建立在算子应用之上，而不必先显式形成完整的 $\widetilde{M}$。

在此基础上，我们采用 randomized SVD 的标准流程 [8]。首先生成高斯随机测试矩阵

$$\Omega \in \mathbb{R}^{n\times \ell}, \qquad \Omega_{ij} \sim \mathcal{N}(0,1),$$

其中 $\ell = k+p$，k 是目标秩的估计规模，p 是 oversampling 参数。然后构造样本矩阵

$$Y_0 = \widetilde{M}\Omega.$$

这里的 Y_0 并不是通过显式矩阵乘法得到，而是通过对 Ω 逐列或成块应用算子 $\widetilde{M}$ 得到。当谱衰减较慢时，为了更稳定地提取主导子空间，还可以做 q 次幂迭代：

$$Y_{t+1} = \widetilde{M}\big(\widetilde{M}^{\top}Y_t\big), \qquad t = 0,1,\ldots,q-1.$$

随后对最终样本矩阵做正交化，得到

$$Q = \text{orth}(Y_q) \in \mathbb{R}^{m\times \ell},$$

其中 Q 近似张成了 $\widetilde{M}$ 的主导列空间。再将原问题压缩到小矩阵

$$M_{proj} = Q^{\top}\widetilde{M} \in \mathbb{R}^{\ell\times n}$$

上，并对其做精确 SVD：

$$M_{proj} = \widetilde{U}\Sigma V^{\top}.$$

到这一步为止，原来的大接口已经被转化成一个小矩阵上的谱截断问题。直接采用固定秩可以直接保留前 r 个奇异方向；而本文主线采用基于尾能量的自适应截断：给定合并容忍度 τ_{merge}，选择满足

$$\left(\frac{\sum_{i>r}\sigma_i^2}{\sum_i \sigma_i^2}\right)^{1/2} \le \tau_{\text{merge}}$$

的最小秩 r。设据此得到的截断结果为

$$\widetilde{M} \approx Q\widetilde{U}_r \Sigma_r V_r^{\top},$$

为了与全文统一的分隔状态表示保持一致，我们将其写回为因子形式

$$A = (Q\widetilde{U}_r)\Sigma_r^{1/2}, \qquad B = V_r \Sigma_r^{1/2},$$

从而得到

$$\widetilde{M} \approx AB^{\top}.$$

这里，自适应秩选择决定“最终保留多少通道”，而随机化压缩决定“如何在不显式收缩大接口的情况下找到这些通道”。

3.6 输出分块、根节点读出与缓存

除了上述核心递归外，完整收缩还需要三个与任务设定直接相关的实现机制。

输出分块。 为了避免一次性面对完整输出空间，我们只对部分开放标签做分块。对固定输出块 Ω_b 而言，若某些开放腿已被限制到较小的局部索引集，则各子树在输出侧的维度也会随之减小。因此，输出分块并不是附加包装，而是直接影响分隔状态规模的组成部分。

根节点读出。 当递归合并到根节点时，由于根节点对应整个网络，其边界标签为空。这时根状态不再是一般的“输出侧 × 边界侧”接口矩阵，而是直接退化为当前输出块的稠密张量。也就是说，分隔状态递归在根节点自然终止，并转化为当前输出块的读出。

块间缓存。 在按块收缩时，不同输出块往往共享大量上游子树。若每个块都从头构建整棵树上的所有状态，那么输出分块的潜在收益会被重复计算抵消。因此，我们对分隔状态做块间缓存。一个状态能否复用，不取决于全局块编号，而取决于该子树真正看到的局部块条件；只要两个输出块在某个子树上的局部切片完全一致，该子树状态就可以直接复用。

3.7 小结

综上，本文方法围绕一个统一对象展开：分隔状态。划分树规定了这些状态如何被组织，递归合并规定了它们如何在树上向上传播，低秩压缩则保证这一传播在计算上可控。输出分块、根节点读出和块间缓存，都服务于这一主线。因此，本文的方法可以概括为

划分树 ＋保持输出边界的分隔状态 ＋递归合并与重压缩.

它近似的不是最终输出，而是在保持输出边界语义不变的前提下，对树上递归传播的内部接口状态做近似。

4 实验

5 结论

A 附录

References

[1] Moses Charikar, Kevin Chen, and Martin Farach-Colton. "Finding Frequent Items in Data Streams". In: *Theoretical Computer Science* 312.1 (2004), pp. 3–15. DOI: 10.1016/S0304-3975(03)00400-6.

[2] Rina Dechter. "Bucket Elimination: A Unifying Framework for Probabilistic Inference". In: *Proceedings of the Twelfth Conference on Uncertainty in Artificial Intelligence*. 1996, pp. 211–219.

[3] Rina Dechter. "Bucket elimination: A unifying framework for probabilistic inference". In: *Learning in graphical models*. Springer, 1998, pp. 75–104.

[4] Rina Dechter and Irina Rish. "Mini-Buckets: A General Scheme for Generating Approximations in Automated Reasoning". In: *Proceedings of the Fifteenth International Joint Conference on Artificial Intelligence*. 1997, pp. 1297–1303.

[5] Eugene F. Dumitrescu, Allison L. Fisher, Timothy D. Goodrich, Travis S. Humble, Blair D. Sullivan, and Andrew L. Wright. "Benchmarking Treewidth as a Practical Component of Tensor Network Simulations". In: *PLOS ONE* 13.12 (2018), e0207827. DOI: 10.1371/journal.pone.0207827.

[6] Johnnie Gray and Garnet Kin-Lic Chan. "Hyperoptimized Approximate Contraction of Tensor Networks with Arbitrary Geometry". In: *Physical Review X* 14.1 (2024), p. 011009. DOI: 10.1103/PhysRevX.14.011009.

[7] Johnnie Gray and Stefanos Kourtis. "Hyper-optimized Tensor Network Contraction". In: *Quantum* 5 (2021), p. 410. DOI: 10.22331/Q-2021-03-15-410.

[8] Nathan Halko, Per-Gunnar Martinsson, and Joel A. Tropp. "Finding Structure with Randomness: Probabilistic Algorithms for Constructing Approximate Matrix Decompositions". In: *SIAM Review* 53.2 (2011), pp. 217–288. DOI: 10.1137/090771806.

[9] Mike Heddes, Igor Nunes, Tony Givargis, and Alex Nicolau. "Approximating Tensor Network Contraction with Sketches". In: *arXiv preprint arXiv:2603.07387* (2026).

[10] Cupjin Huang, Fang Zhang, Michael Newman, Xiaotong Ni, Dawei Ding, Junjie Cai, Xun Gao, Tenghui Wang, Feng Wu, Gengyan Zhang, et al. "Efficient Parallelization of Tensor Network Contraction for Simulating Quantum Computation". In: *Nature Computational Science* 1 (2021), pp. 578–587. DOI: 10.1038/s43588-021-00119-7.

[11] Jonathan Jakes-Schauer, David Anekstein, and Pawel Wocjan. "Carving-width and Contraction Trees for Tensor Networks". In: *arXiv preprint arXiv:1908.11034* (2019).

[12] Michael Levin and Cody P. Nave. "Tensor Renormalization Group Approach to Two-Dimensional Classical Lattice Models". In: *Physical Review Letters* 99.12 (2007), p. 120601. DOI: 10.1103/PhysRevLett.99.120601.

[13] Linjian Ma, Matthew T. Fishman, E. Miles Stoudenmire, and Edgar Solomonik. "Approximate Contraction of Arbitrary Tensor Networks with a Flexible and Efficient Density Matrix Algorithm". In: *Quantum* 8 (2024), p. 1580. DOI: 10.22331/q-2024-12-27-1580.

[14] Igor L. Markov and Yaoyun Shi. "Simulating Quantum Computation by Contracting Tensor Networks". In: *SIAM Journal on Computing* 38.3 (2008), pp. 963–981. DOI: 10.1137/050644756.

[15] Román Orús. "A Practical Introduction to Tensor Networks: Matrix Product States and Projected Entangled Pair States". In: *Annals of Physics* 349 (2014), pp. 117–158. DOI: 10.1016/j.aop.2014.06.013.

[16] Feng Pan, Pengfei Zhou, Sujie Li, and Pan Zhang. "Contracting Arbitrary Tensor Networks: General Approximate Algorithm and Applications in Graphical Models and Quantum Circuit Simulations". In: *Physical Review Letters* 125.6 (2020), p. 060503. DOI: 10.1103/PhysRevLett.125.060503.

[17] Ninh Pham and Rasmus Pagh. "Fast and Scalable Polynomial Kernels via Explicit Feature Maps". In: *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2013, pp. 239–247. DOI: 10.1145/2487575.2487591.